迷你编码智能体实践报告

学号：24371220 模型：deepseek-flash

一、总体实现

本次作业实现了一个命令行编码智能体。程序读取题面和代码仓库后，向 DeepSeek 发送上下文；模型每轮返回一个 JSON 动作，程序解析并执行，再把结果交给模型。整个循环自行编写，没有使用现成 Agent 框架。

实现时把模型决策和底层执行分开：agent.py 组织消息、维护计划并控制循环，runtime.py 处理 API、文件、命令、日志和补丁，context_selection.py 实现特色上下文策略。核心合计 288 个非空、非纯注释物理行，包含 SYSTEM、Agent 和整个上下文选择模块；CLI 和底层操作另计。

二、四项基础功能

用户交互采用命令行多轮输入。用户可以追加要求、查看当前计划或中断执行；写文件和运行命令前，默认展示动作并请求确认。评测时使用 --auto，程序不再等待键盘输入。

基线的上下文管理采用头尾裁剪和滑动窗口。完整题面、追加要求和当前状态固定保留；单条结果超过 4500 字符时保留头尾，请求内容超过 15000 字符时删除最早的动作及结果。这种办法简单，但可能丢掉长输出中段的错误，也可能忘记较早读过的关键代码。

任务管理使用 todo、doing、done 三种状态。模型先拆分子目标，再更新进度；程序限制计划标题和顺序不能改变。这样的约束能保留原始目标，但并不能强制模型按正确顺序完成每一步，第四题的重复评测也暴露了这一点。

工具至少支持完整读文件、写文件和执行命令，另提供文件列表与计划更新。写入范围从题面提取，只允许修改指定业务文件；命令只支持完整 pytest 和题目提供的 run_checks.py。工具的通用性有限，但可以减少误改测试和越界操作。

三、运行记录与验证

run.log 随真实操作逐条保存完整请求、响应、文件内容和命令输出。发给模型的内容可以裁剪，原始日志不裁剪。patch.diff 由初始快照和最终代码生成；即使运行失败或被中断，也保存已有改动。结束前要求修改后的测试与诊断通过，且计划全部完成。

环境为 Ubuntu WSL 2、Python 3.12.3、pytest 8.4.2 和 GNU patch 2.7.6。12 项机制测试通过，覆盖路径限制、命令限制、交互中断、上下文预算及诊断状态更新。公开题成绩和特色实验结果见第三页。

特色功能的设计与实验方法

Diagnostic-Driven Context Selection and Compression

四、为什么增加诊断选择

基线只按长度和时间裁剪信息，不判断内容是否与当前错误有关。长日志里的关键断言如果恰好位于中段，就可能被删掉。因此，特色功能尝试先根据执行反馈判断当前问题，再选择下一轮模型需要的内容。第一版使用确定性规则，不增加额外模型调用。

五、选择规则与实现范围

工具结果先存入候选池。同一命令的新结果替换旧结果，成功写入后移除该文件先前读到的版本。程序结合最近写入和验证结果判断当前阶段，优先信息如下表。

状态	判断依据	优先信息
探索定位	尚未形成有效诊断	题面、文件列表、已读业务代码
修复失败	当前 pytest 或诊断返回失败	失败测试、异常位置、相关代码
修改后验证	已写入，尚未完成测试验证	最近改动、待验证项、历史失败参考
收尾	最近完整测试已通过	验证结果、计划完成状态

程序从失败输出中提取文件名、测试名和异常类型，给当前验证、失败结果、路径匹配和近期代码加分，再按分数填入剩余预算。长命令输出保留断言、失败项和汇总附近原文，合并重复行；仍过长时再裁剪头尾。识别不出诊断结构时也退回头尾裁剪。

完整题面和当前计划仍固定保留，所有原始输出保存在日志中。context_summary 另记候选分数、选择原因和实际发送内容。当前版本没有函数级切片或跨文件依赖分析，状态判断也只用于排序，不能代替测试与计划的完成检查。

六、对照实验设置

主实验固定第五题，两种模式交替各运行 3 次，每次使用新副本；随后两种模式各跑一次全部五题。两组共享代码、提示词和工具，仅通过 --context-mode baseline|diagnostic 切换上下文策略。

固定模型 deepseek-flash、温度 0、上下文 15000 字符、单次输出 4096 token、最多 60 步；智能体上限 600 秒，最终验证上限 120 秒。比较整题通过情况、API 上报的输入 token、调用次数和耗时，全部运行记录均保留。

实验结果与分析

共 16 次真实运行；以下结果区分功能测试与题目过程要求。

七、第五题的三轮对照

轮次	输入 token：基线 / 诊断	调用次数：基线 / 诊断	耗时秒：基线 / 诊断
1	60,648 / 79,956	15 / 28	36.8 / 89.8
2	83,301 / 73,170	19 / 29	42.4 / 84.2
3	71,984 / 69,161	15 / 20	30.7 / 62.8

平均输入量：基线 71,978 token，诊断 74,096 token（+2.9%）。诊断模式三轮调用均更多，本次未观察到稳定的成本收益。输入 token 包含重复上下文，并不等于实际费用。

八、公开题成绩及第四题说明

本次五题回归中，两组的 86 个功能测试均通过。按课程整题评分规则，基线为 4/5，诊断模式为 5/5。基线失分的是第四题的执行过程，不是某一个功能测试失败。

第四题要求先单独修复净额，运行完整测试确认净额正确，并将第一项计划标记为 done，然后才能实现 CSV。这次基线将两项修改合并写入，跳过了中间验证和相应的计划更新，因此尽管最终 10 个测试全过，过程检查仍判为不通过。其余四题通过。

此前保存的基线 attempt05 为 5/5，这次重复运行得到 4/5，说明模型遵守步骤的表现会波动。诊断模式本次通过五题，但单轮回归不能证明它更稳定，也不能把此次得分差异直接归因于上下文选择。

九、结果的适用范围

课程仓库较小、修复点集中，基线可能已保住足够线索，任务难度未必能体现诊断选择的优势。不过，诊断模式确实出现了更多重复调用，重组历史也可能削弱动作和结果的连贯性；不能把未获收益全部归因于题目简单。

后续可预先设计含无关日志、中段关键错误和多文件依赖的压力任务，再在相同预算下比较。温度 0 不保证完全复现，这三轮结果只支持小样本观察。

证据：experiments/feature/comparison01/ 下的 protocol.json、results.json 及各轮配置、完整日志、补丁。参考：课程 README、题目、日志规范和公开评测器。实现与报告由 AI 编程助手协助完成，数字来自真实 API 运行。